Heap Sort -

Concept:

- Based on Binary Heap (Complete Binary Tree).
- First, build a max-heap → largest element at the root.
- Swap root with last element & reduce heap size.
- Heapify again to maintain max-heap property.
- Repeat until array is sorted.

Max-heap:

- Every parent node is greater than or equal to its children.

  → So, the largest element is always at the root (arr[0]).

- Result using Max-heap: the array gets sorted in ascending order.

Heap Sort -

Algorithm (Step-by-step):

- Build a max-heap from the array.
- Swap the root (max) with the last element.
- Reduce heap size by 1.
- Heapify the root element.
- Repeat until heap size = 1.

Time Complexity:

- Best case: O(n log n)
- Worst case: O(n log n)
- Average case: O(n log n)
- Space Complexity: O(1) (in-place sorting)

Heap Sort -

Ques. Sort an array using Heap Sort.

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

Heap Sort -

Ques. Sort an array using Heap Sort.

```
def heap_sort(arr):
    n = len(arr)

    # Build max-heap
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

# Example
arr = [12, 11, 13, 5, 6, 7]
print("Sorted Array:", heap_sort(arr))
```